

# 人工智能期末复习课详细提纲

## Contents

|          |                                   |           |
|----------|-----------------------------------|-----------|
| <b>1</b> | <b>全连接网络与残差网络基础</b>               | <b>2</b>  |
| 1.1      | 单个神经元与权重                          | 3         |
| 1.2      | 全连接层                              | 3         |
| 1.3      | 多层神经网络与激活函数                       | 4         |
| 1.4      | 深层网络训练困难                          | 5         |
| 1.5      | 残差连接                              | 5         |
| 1.6      | 关键记忆                              | 6         |
| 1.7      | 思考题                               | 7         |
| <b>2</b> | <b>卷积神经网络 CNN</b>                 | <b>7</b>  |
| 2.1      | 图像数据的特征：关联性                       | 7         |
| 2.2      | 图像数据的特征：不变性                       | 7         |
| 2.3      | 大脑如何处理图像：类视皮层设计                   | 7         |
| 2.4      | 卷积运算                              | 8         |
| 2.5      | Zero Padding                      | 9         |
| 2.6      | Stride                            | 10        |
| 2.7      | 多通道卷积                             | 12        |
| 2.8      | 卷积层尺寸计算总结                         | 12        |
| 2.9      | 池化 Pooling                        | 13        |
| 2.10     | CNN 整体结构与层的组合                     | 14        |
| 2.11     | 关键记忆                              | 14        |
| 2.12     | 思考题                               | 14        |
| <b>3</b> | <b>循环神经网络 RNN 与 LSTM</b>          | <b>15</b> |
| 3.1      | 自然语言处理 NLP 的任务特点                  | 15        |
| 3.2      | 数据准备                              | 16        |
| 3.3      | 分词、one-hot 与 embedding            | 16        |
| 3.4      | Next Token Prediction             | 17        |
| 3.5      | RNN 的设计                           | 18        |
| 3.6      | BPTT 与梯度问题                        | 20        |
| 3.7      | LSTM                              | 21        |
| 3.8      | 关键记忆                              | 22        |
| 3.9      | 思考题                               | 22        |
| <b>4</b> | <b>Transformer</b>                | <b>22</b> |
| 4.1      | 为什么需要 Transformer                 | 22        |
| 4.2      | Next Token Prediction 框架          | 23        |
| 4.3      | Embedding 与位置编码                   | 24        |
| 4.4      | 注意力机制与 Q、K、V                      | 25        |
| 4.5      | Causal Mask                       | 27        |
| 4.6      | Transformer Block、LayerNorm 与 FNN | 27        |
| 4.7      | 输出投影与解码                           | 28        |
| 4.8      | 关键记忆                              | 29        |
| 4.9      | 思考题                               | 29        |

|           |                       |           |
|-----------|-----------------------|-----------|
| <b>5</b>  | <b>神经网络训练现象：频率原则</b>  | <b>30</b> |
| 5.1       | 过参数化与隐式偏好             | 30        |
| 5.2       | 频率、低频与高频              | 30        |
| 5.3       | 频率原则                  | 30        |
| 5.4       | 早停 Early stopping     | 31        |
| 5.5       | 思考题                   | 31        |
| <b>6</b>  | <b>参数凝聚与解的平坦性</b>     | <b>31</b> |
| 6.1       | 初始化大小的影响              | 31        |
| 6.2       | 参数凝聚                  | 31        |
| 6.3       | 平坦解与尖锐解               | 32        |
| 6.4       | 思考题                   | 32        |
| <b>7</b>  | <b>大模型介绍</b>          | <b>32</b> |
| 7.1       | 什么是大模型                | 32        |
| 7.2       | Encoder、Decoder 与常见架构 | 33        |
| 7.3       | 预训练、微调、提示词、蒸馏         | 33        |
| 7.4       | 大模型中的现象               | 33        |
| 7.5       | 思考题                   | 34        |
| <b>8</b>  | <b>强化学习</b>           | <b>34</b> |
| 8.1       | 强化学习是什么               | 34        |
| 8.2       | 状态、观察、动作空间、奖励         | 34        |
| 8.3       | 回报与折扣因子               | 35        |
| 8.4       | 情节型任务与连续型任务           | 35        |
| 8.5       | 探索与利用                 | 35        |
| 8.6       | 策略、价值函数与最优策略          | 36        |
| 8.7       | MDP 与马尔可夫性            | 36        |
| 8.8       | Bellman 方程的直观含义       | 36        |
| 8.9       | DP、MC 与深度强化学习         | 37        |
| 8.10      | 强化学习应用                | 37        |
| 8.11      | 思考题                   | 38        |
| <b>9</b>  | <b>最后一遍串讲清单</b>       | <b>38</b> |
| 9.1       | 残差网络必会                | 38        |
| 9.2       | CNN 必会                | 38        |
| 9.3       | RNN/LSTM 必会           | 38        |
| 9.4       | Transformer 必会        | 38        |
| 9.5       | 训练现象必会                | 39        |
| 9.6       | 大模型必会                 | 39        |
| 9.7       | 强化学习必会                | 39        |
| <b>10</b> | <b>不建议作为考试重点的内容</b>   | <b>39</b> |
| <b>11</b> | <b>两节复习课建议讲法</b>      | <b>40</b> |
| 11.1      | 第一节课：网络结构主线           | 40        |
| 11.2      | 第二节课：训练现象、大模型、强化学习    | 40        |

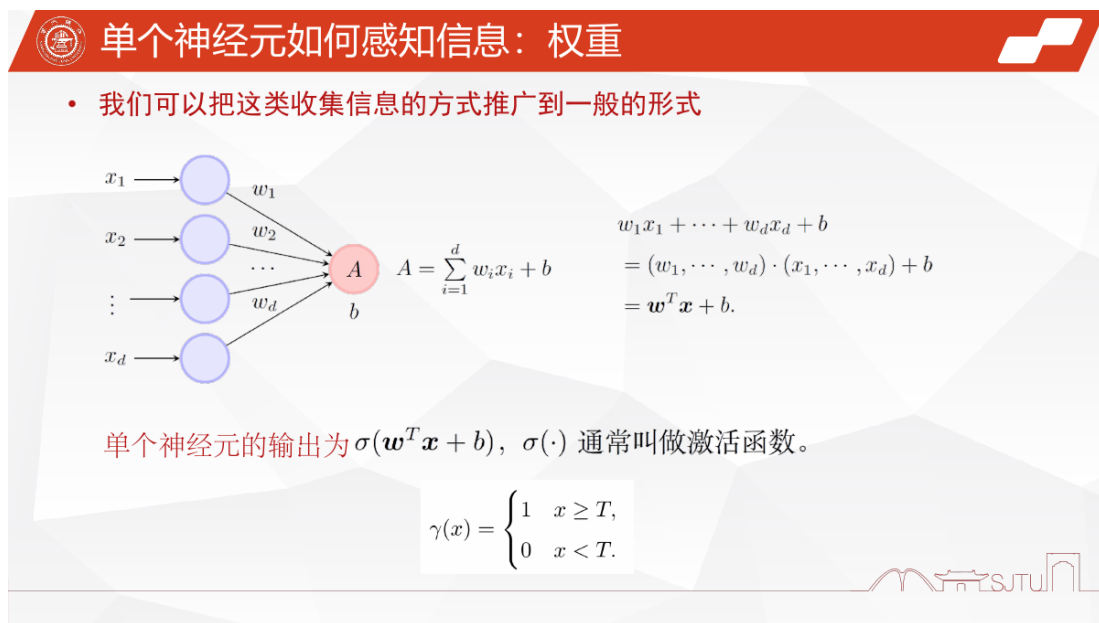
## 1 全连接网络与残差网络基础

这一章按 PDF 顺序复习：单个神经元 -> 全连接层 -> 多层神经网络 -> 深层网络训练困难 -> 残差连接。第 11-12 节中，全连接网络只保留理解后续章节所需的基础；复习重点放在残差网络为什么出现、残差连接怎么写、它解决什么问题。

## 1.1 单个神经元与权重

页码: p4-p6

单个神经元可以理解为“收集多个输入，按重要性加权，再经过激活函数输出”。权重越大，说明对应输入对输出影响越大。



神经元：神经元把输入按权重加权求和，加上 bias，再通过激活函数得到输出。

单个神经元公式：

$$z = \mathbf{w}^T \mathbf{x} + b, \quad y = \sigma(z)$$

其中， $\mathbf{x}$  是输入向量， $\mathbf{w}$  是权重向量， $b$  是 bias， $\sigma$  是激活函数， $y$  是神经元输出。

例子 1：单个神经元计算。设  $\mathbf{x} = [2, 1]^T$ ， $\mathbf{w} = [0.5, -1]^T$ ， $b = 0.2$ ，则

$$z = 0.5 \times 2 + (-1) \times 1 + 0.2 = 0.2$$

神经元最后输出为  $y = \sigma(0.2)$ 。如果  $\sigma$  是 ReLU，则  $y = \max(0, 0.2) = 0.2$ 。

## 1.2 全连接层

页码: p7-p8

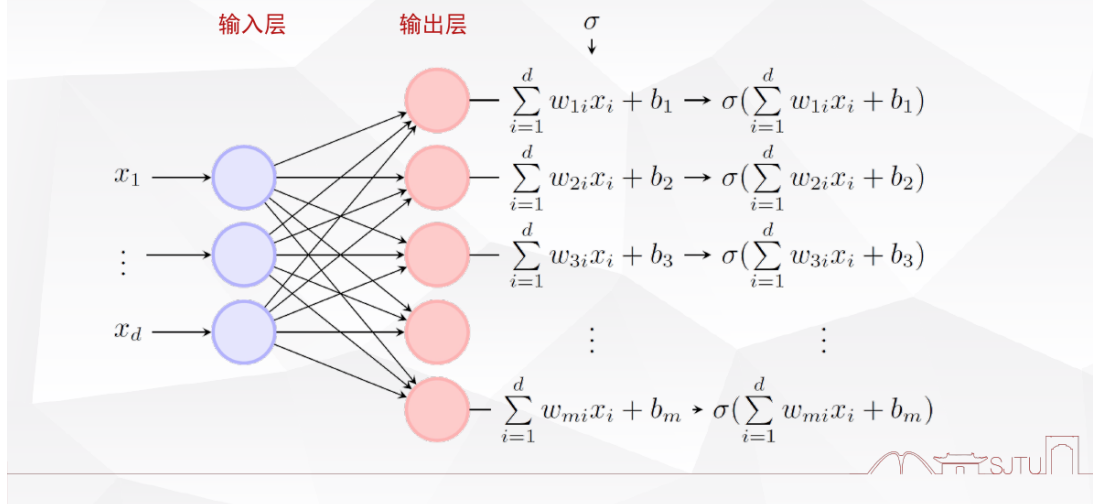
把多个神经元放在同一层，并让每个输出神经元都连接到所有输入，就得到全连接层。全连接层的核心是矩阵乘法。



## 单层神经网络



- 简单地把多个神经元堆在一起，我们就可以得到单层神经网络



全连接层：全连接层中，每个输出神经元都接收上一层所有输入；它可以写成矩阵形式  $y = \sigma(Wx + b)$ 。

全连接层公式：

$$y = \sigma(Wx + b)$$

如果输入维度是  $d_{in}$ ，输出维度是  $d_{out}$ ，则

$$x \in R^{d_{in}}, \quad W \in R^{d_{out} \times d_{in}}, \quad b \in R^{d_{out}}, \quad y \in R^{d_{out}}$$

例子 2：全连接层参数量。输入维度  $d_{in} = 4$ ，输出维度  $d_{out} = 3$ 。权重矩阵  $W$  有  $3 \times 4 = 12$  个参数，bias 有 3 个参数，所以总参数数量为  $12 + 3 = 15$ 。

### 1.3 多层神经网络与激活函数

页码：p10-p18

多层神经网络是在全连接层基础上继续堆叠隐藏层。只做线性变换不够，因为很多任务本身是非线性的；加入非线性激活函数后，网络才能表达更复杂的函数。

多层神经网络 **MLP**：MLP 由多层全连接层和非线性激活函数堆叠而成，常见形式是“线性变换 + 激活函数”重复多次。

一层隐藏层的常见形式：

$$h = \sigma(W_1x + b_1), \quad y = W_2h + b_2$$

其中， $h$  是隐藏层表示。考试重点不是复杂推导，而是知道：激活函数让网络具有非线性表达能力。

例子 3：为什么需要激活函数。如果连续两层都只是线性变换，那么

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$$

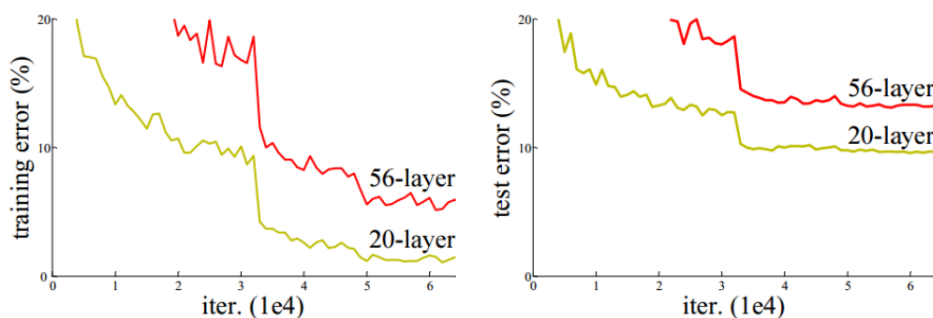
整体仍然只是一个线性变换。加入非线性激活函数后，多层网络才不容易退化成单层线性模型。

## 1.4 深层网络训练困难

页码：p19-p21

网络变深后，理论表达能力更强，但实际训练不一定更好。普通深层网络可能出现梯度消失、梯度爆炸和退化问题。

### 退化问题



利用20层和56层的普通网络对于CIFAR-10数据集的训练误差（左图）和测试误差（右图）。可见越深的网络会产生越大的训练误差和测试误差。

[1] He, Kaiming, et al. "Deep Residual Learning for Image Recognition." IEEE (2016).



退化问题：当网络加深后，训练误差和测试误差没有变好，反而可能变差，这说明深层普通网络更难优化。

需要区分两件事：

| 问题              | 含义                            | 记忆                              |
|-----------------|-------------------------------|---------------------------------|
| 梯度消失/爆炸<br>退化问题 | 反向传播时梯度过小或过大<br>网络更深后训练误差反而更高 | 深层网络训练不稳定<br>不是简单的表达能力不足，而是优化困难 |

Sigmoid 函数在输入很大或很小时会饱和，梯度接近 0。深层网络反向传播要经过很多层，梯度可能不断变小，导致前面层的参数很难更新。

例子 4：梯度消失的直观理解。如果每一层反向传播时梯度都乘以 0.5，经过 10 层后大约变成  $0.5^{10} \approx 0.001$ 。梯度太小，前面层就学得很慢。

## 1.5 残差连接

页码：p22-p23

残差网络的核心做法是在普通网络层旁边加入一条 shortcut，让输出同时包含新学习到的变化和原始输入信息。

## 残差神经网络



问题：随着网络层数的增加，训练深层网络越来越困难

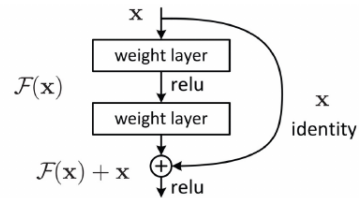
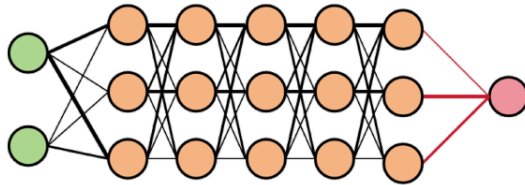
□ 梯度爆炸和梯度消失问题

□ 退化问题：网络深度增加时，性能出现饱和甚至下降的现象

□ 额外增加的层可能并未学习到新的特征表示，而只是近似恒等映射

□ 网络越深，参数量越大，过拟合风险也随之增加，导致泛化性能下降

解决方案：加入残差连接： $F_{l+1} = \sigma(W_l F_l + b_l) + F_l$



[1] He, Kaiming, et al. "Deep Residual Learning for Image Recognition." IEEE (2016).



残差连接：残差连接把某一层的输入  $x$  直接加到变换结果  $F(x)$  上，使模块输出变为  $F(x) + x$ 。

残差连接的基本形式：

$$y = F(x) + x$$

课件中也写成：

$$F_{l+1} = \sigma(W_l F_l + b_l) + F_l$$

其中， $F_l$  是第  $l$  层输入特征， $\sigma(W_l F_l + b_l)$  是这一层学习到的变换， $+F_l$  表示把输入信息直接加回来。

例子 5：残差输出计算。如果输入  $x = [1, 2]^T$ ，网络学习到的变化是  $F(x) = [0.2, -0.1]^T$ ，则残差模块输出为

$$F(x) + x = [0.2, -0.1]^T + [1, 2]^T = [1.2, 1.9]^T$$

残差连接的意义不是让网络“少学一点”，而是让网络更容易学习恒等映射。如果某些额外层暂时学不到有用特征，它们可以让  $F(x)$  接近 0，此时输出接近  $x$ ，网络不会因为加深而明显变差。

例子 6：恒等映射。如果一个残差模块暂时没有学到有效变化，令  $F(x) \approx 0$ ，则输出  $F(x) + x \approx x$ 。这说明深层网络至少可以比较容易地保留原来的信息。

### 1.6 关键记忆

- 单个神经元的基本计算是  $z = w^T x + b$ ，再经过激活函数得到输出。p4-p6
- 全连接层可以写成  $y = \sigma(Wx + b)$ ；如果输入维度为  $d_{in}$ 、输出维度为  $d_{out}$ ，则  $W \in R^{d_{out} \times d_{in}}$ 。p7-p8
- 激活函数提供非线性能力；如果多层网络只有线性变换，整体仍然等价于一个线性变换。p10-p18
- 深层普通网络可能遇到梯度消失、梯度爆炸和退化问题；网络更深不一定更容易训练。p19-p21
- 残差连接的基本形式是  $F(x) + x$ ，它帮助信息和梯度在深层网络中传播。p22-p23
- ResNet 主要缓解深层网络训练困难和退化问题，不是单纯用于减少参数或解决过拟合。p22-p23

## 1.7 思考题

1. 填空：单个神经元常见计算形式是  $z = w^T x + b$ ，其中  $b$  叫做 \_\_\_\_。  
答案：bias，也叫偏置项。
2. 判断：如果多层网络中每一层都只有线性变换，那么多层叠加后整体仍然等价于一个线性变换。  
答案：正确。
3. 计算：输入维度为 5、输出维度为 2 的全连接层，如果包含 bias，总参数量是多少？  
答案： $2 \times 5 + 2 = 12$ 。
4. 填空：残差连接的基本形式可以写成  $F(x) + \_\_\_\_\_\_$ 。  
答案： $x$ 。
5. 判断：残差连接本身的核心作用是让信息和梯度更容易在深层网络中传播。  
答案：正确。

## 2 卷积神经网络 CNN

这一章按 PDF 顺序复习：图像特性 -> 类视皮层启发 -> 卷积计算 -> padding -> stride -> 多通道卷积 -> 池化 -> CNN 结构。页码均指本章 PDF 页码。

### 2.1 图像数据的特征：关联性

页码：p3-p5

图像有空间结构。相邻像素通常更相关，距离越远，关联通常越弱。

图像局部关联性：图像中距离较近的像素或局部区域通常更相关，距离越远，相关性通常越弱。CNN 的局部连接设计正是为了利用这种特点。

| 模型    | 形式                  | 记忆           |
|-------|---------------------|--------------|
| 指数衰减  | $y = e^{-kx}$       | 下降较快         |
| 幂级数衰减 | $y = \frac{1}{x^k}$ | 下降较慢，长程关联更明显 |

幂级数衰减可以看作许多指数衰减成分的叠加：

$$\int_0^\infty e^{-\alpha t} d\alpha = \frac{1}{t}$$

### 2.2 图像数据的特征：不变性

页码：p6

图像识别不应只记住像素位置。物体轻微平移、局部扰动或光照稍变时，类别通常不应改变。

不变性：输入发生小幅平移、扰动或光照变化时，模型仍应尽量识别出相同语义类别。

### 2.3 大脑如何处理图像：类视皮层设计

页码：p7-p10

图像处理可以理解为：先提取局部简单特征，再逐步整合成复杂信息。

| 类比对象 | 作用          | CNN 中的对应模块  |
|------|-------------|-------------|
| 简单细胞 | 对位置、边缘、方向敏感 | 卷积层         |
| 复杂细胞 | 整合多个局部输入    | 池化、全连接等后续模块 |

| 类比对象 | 作用 | CNN 中的对应模块 |
|------|----|------------|
|------|----|------------|

| CNN 模块 | 作用      |
|--------|---------|
| 卷积层    | 提取局部特征  |
| 池化层    | 汇总局部信息  |
| 全连接层   | 综合特征    |
| 输出层    | 输出类别或概率 |

## 2.4 卷积运算

页码: p11-p13

卷积核是一个小窗口，例如  $3 \times 3$ 。它在图像上滑动，每次覆盖一个局部区域，把局部区域和卷积核对应位置相乘后求和。

卷积运算：卷积核在输入图像或特征图上滑动，每次对局部窗口做对应位置相乘再求和，得到输出特征图中的一个值。

### 卷积运算



右边的例子中用的是  $g(-u, -v)$ ，是否等价？

卷积核

是等价，本质上，做个变量代换即可。

$$F(x, y) = \sum_{u, v} f(x - u, y - v) g(u, v)$$

$$F(x, y) = \int f(x - u, y - v) g(u, v) du dv$$

|             |            |            |
|-------------|------------|------------|
| $g(-1, 1)$  | $g(0, 1)$  | $g(1, 1)$  |
| $g(-1, 0)$  | $g(0, 0)$  | $g(1, 0)$  |
| $g(-1, -1)$ | $g(0, -1)$ | $g(1, -1)$ |

|  |              |            |              |  |
|--|--------------|------------|--------------|--|
|  |              |            |              |  |
|  | $(x-1, y+1)$ | $(x, y+1)$ | $(x+1, y+1)$ |  |
|  | $(x-1, y)$   | $(x, y)$   | $(x+1, y)$   |  |
|  | $(x-1, y-1)$ | $(x, y-1)$ | $(x+1, y-1)$ |  |
|  |              |            |              |  |

$$F(x, y) = f(x-1, y+1)g(-1, 1) + f(x, y+1)g(0, 1) + f(x+1, y+1)g(1, 1) \\ + f(x-1, y)g(-1, 0) + f(x, y)g(0, 0) + f(x+1, y)g(1, 0) \\ + f(x-1, y-1)g(-1, -1) + f(x, y-1)g(0, -1) + f(x+1, y-1)g(1, -1)$$

[1] 《深度学习现象导论》许志钦、张耀宇 [https://github.com/xuzhiqin1990/understanding\\_dl](https://github.com/xuzhiqin1990/understanding_dl)



单通道卷积例子：

图像局部区域  $X$ ：

$$\begin{bmatrix} 1 & 2 & 0 \\ 3 & 1 & 2 \\ 0 & 1 & 1 \end{bmatrix}$$

卷积核  $K$ ：

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

一次卷积输出为

$$1 \times 1 + 2 \times 0 + 0 \times (-1) + 3 \times 1 + 1 \times 0 + 2 \times (-1) + 0 \times 1 + 1 \times 0 + 1 \times (-1) = 1$$



卷积核继续向右或向下滑动，就得到下一个位置的输出。所有位置的输出组成新的特征图。

| 模型  | 连接方式          | 对图像结构的利用   |
|-----|---------------|------------|
| MLP | 常把图像展平成向量，全连接 | 容易弱化二维空间结构 |
| CNN | 局部连接，卷积核滑动    | 直接利用局部关联   |

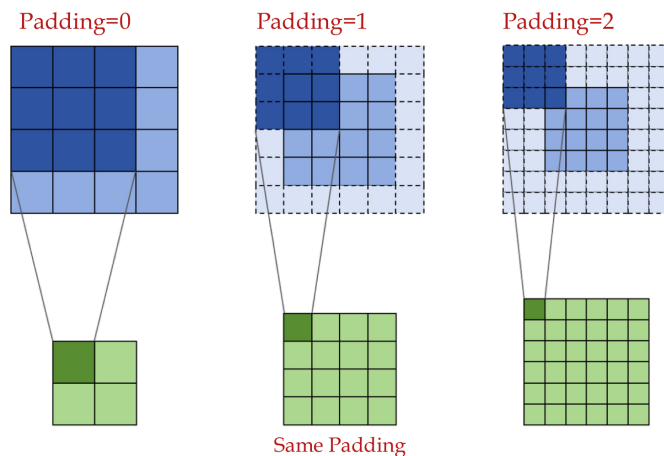
## 2.5 Zero Padding

页码：p14

Padding 是在图像边缘补值，常见是补 0。作用是控制卷积后图像大小，避免图像尺寸过快变小。

**Padding：** Padding 是在输入边缘补值，常见是补 0，用来控制卷积后的空间尺寸，并让边缘位置也能参与更多卷积计算。

## Zero Padding——解决图像过卷积后大小改变的问题



[1] 《深度学习现象导论》许志钦、张耀宇 [https://github.com/xuzhiqin1990/understanding\\_dl](https://github.com/xuzhiqin1990/understanding_dl)



**Zero padding 例子：**原图像为  $3 \times 3$ ：

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

padding 为 1 后变成  $5 \times 5$ ：

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 3 & 0 \\ 0 & 4 & 5 & 6 & 0 \\ 0 & 7 & 8 & 9 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

使用  $3 \times 3$  卷积核、stride=1 时：

不加 padding:  $3 \times 3 \rightarrow 1 \times 1$ 。加 padding 为 1:  $3 \times 3 \rightarrow 3 \times 3$ 。

## 2.6 Stride

页码: p15

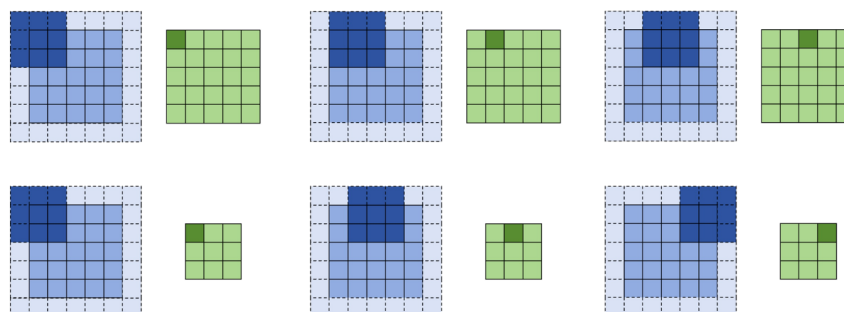
Stride 是卷积核滑动步长。课件用  $\text{stride}(i, j)$  表示横向跨  $i$  步、纵向跨  $j$  步。

**Stride:** Stride 是卷积核每次滑动的步长。stride 越大，卷积核采样位置越少，输出空间尺寸通常越小。

## 跨步 stride(i, j)



横向跨i步，纵向跨j步。图中第一行为stride(1,1)，第二行为stride(2,2)



[1] 《深度学习现象导论》许志钦、张耀宇 [https://github.com/xuzhiqin1990/understanding\\_dl](https://github.com/xuzhiqin1990/understanding_dl)



**Stride 例子：**输入长度方向位置为 1, 2, 3, 4, 5，卷积核大小为 3。  
stride 为 1 时，窗口为 [1, 2, 3]、[2, 3, 4]、[3, 4, 5]，输出长度为 3。  
stride 为 2 时，窗口为 [1, 2, 3]、[3, 4, 5]，输出长度为 2。

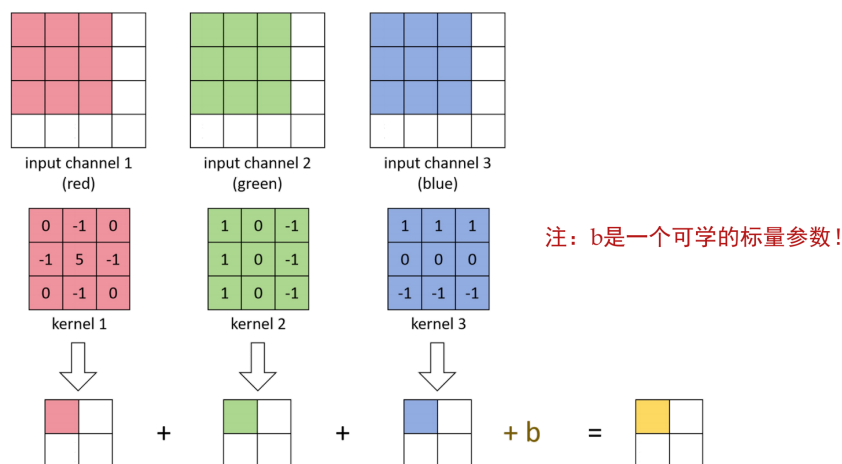
## 2.7 多通道卷积

页码: p16

RGB 图像有 3 个输入通道。多通道卷积中，一个完整卷积核需要覆盖所有输入通道。

多通道卷积：对于有  $C_{in}$  个输入通道的数据，一个完整卷积核的深度也必须覆盖  $C_{in}$  个通道；一个卷积核产生一个输出通道。

### 多通道卷积



[1] 《深度学习现象导论》许志钦、张耀宇 [https://github.com/xuzhiqin1990/understanding\\_dl](https://github.com/xuzhiqin1990/understanding_dl)



多通道卷积例子：RGB 输入图像有 3 个通道。R 通道局部区域与卷积核 R 部分相乘求和，G 通道局部区域与卷积核 G 部分相乘求和，B 通道局部区域与卷积核 B 部分相乘求和；三个通道结果相加，再加 bias，就是这个卷积核在当前位置的输出。

通道数例子：输入为  $32 \times 32 \times 3$ ，一个完整卷积核大小为  $5 \times 5 \times 3$ 。如果卷积核个数为 6，则输出空间大小另算，输出通道数为 6。

## 2.8 卷积层尺寸计算总结

页码: p14-p16

卷积层的计算重点记三件事：空间大小怎么变，通道数怎么变，参数量怎么算。

卷积层输出形状：输入为  $H \times W \times C_{in}$ ，卷积核个数为  $C_{out}$  时，输出形状为  $H_{out} \times W_{out} \times C_{out}$ ；输出通道数由卷积核个数决定。

输入、卷积核和输出的形状：

输入:  $H \times W \times C_{in}$

一个卷积核:  $K \times K \times C_{in}$

卷积核个数:  $C_{out}$

输出:  $H_{out} \times W_{out} \times C_{out}$

符号含义：H, W 是输入高和宽；C\_in 是输入通道数；K 是卷积核大小，例如 3 x 3 卷积核中 K = 3；C\_out 是卷积核个数，也就是输出通道数；H\_out, W\_out 是输出高和宽。

输出空间尺寸公式：

$$H_{out} = \text{floor}((H + 2P - K) / S) + 1$$

$$W_{out} = \text{floor}((W + 2P - K) / S) + 1$$

其中，P 是 padding 大小，S 是 stride 大小，floor 表示向下取整。

padding、stride、卷积核大小的影响：

| 参数      | 增大后通常会怎样 | 直观理解            |
|---------|----------|-----------------|
| padding | 输出空间尺寸变大 | 边缘补了一圈，能滑动的位置更多 |
| stride  | 输出空间尺寸变小 | 卷积核跳得更远，采样位置更少  |
| 卷积核大小   | 输出空间尺寸变小 | 窗口更大，可放置的位置更少   |
| 卷积核个数   | 输出通道数变多  | 一个卷积核产生一个输出通道   |

参数量计算：

卷积层通常有 bias。每个卷积核产生一个输出通道，因此每个输出通道对应 1 个 bias。

不考虑 bias：

$$\text{参数量} = K \times K \times C_{in} \times C_{out}$$

考虑 bias：

$$\text{参数量} = K \times K \times C_{in} \times C_{out} + C_{out}$$

例子 1：参数量计算。输入通道数  $C_{in} = 3$ ，卷积核大小  $K = 5$ ，卷积核个数  $C_{out} = 6$ 。不考虑 bias 时，参数量为  $5 \times 5 \times 3 \times 6 = 450$ ；考虑 bias 时，参数量为  $5 \times 5 \times 3 \times 6 + 6 = 456$ 。

例子 2：padding 保持大小。输入为  $32 \times 32 \times 3$ ，使用  $3 \times 3$  卷积核，卷积核个数为 16，padding 为 1，stride 为 1。

$$H_{out} = \frac{32 + 2 \times 1 - 3}{1} + 1 = 32$$

因此输出为  $32 \times 32 \times 16$ 。

例子 3：stride 让尺寸减小。输入为  $32 \times 32 \times 3$ ，使用  $3 \times 3$  卷积核，卷积核个数为 16，padding 为 1，stride 为 2。

$$H_{out} = \left\lfloor \frac{32 + 2 \times 1 - 3}{2} \right\rfloor + 1 = 16$$

因此输出为  $16 \times 16 \times 16$ 。

## 2.9 池化 Pooling

页码：p18

池化层用于对局部区域做汇总，常见方式是最大池化和平均池化。池化通常用于降低空间尺寸、减少计算量，一般没有可学习参数，也通常不改变通道数。

**池化 Pooling**：池化是在局部窗口内做汇总操作。最大池化取窗口最大值，保留最强响应；平均池化取窗口平均值，保留整体平均水平。

池化例子：对局部区域

$$\begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$$

做最大池化时，输出为  $\max(1, 3, 2, 4) = 4$ ；做平均池化时，输出为  $\frac{1+3+2+4}{4} = 2.5$ 。

尺寸例子：输入为  $32 \times 32 \times 16$ ，最大池化窗口为  $2 \times 2$ ，stride 为 2，输出为  $16 \times 16 \times 16$ 。

## 2.10 CNN 整体结构与层的组合

页码：p20-p22

p20 给出了一个典型 CNN 结构：

输入 RGB 三通道图像

-> 卷积 + ReLU + 池化

-> 卷积 + ReLU + 池化

-> 展平

-> 全连接

-> Softmax 输出类别概率

经典 LeNet 结构：输入图像 → 卷积 → 池化 → 卷积 → 池化 → 展平 → 全连接 → 输出。

需要注意：CNN 没有唯一固定的网络格式。卷积层、激活函数、池化层、全连接层等都可以按任务需要组合。

常见组合方式：

卷积 -> 激活 -> 卷积 -> 激活 -> 池化

卷积 -> 激活 -> 池化 -> 卷积 -> 激活 -> 池化

卷积 -> 激活 -> 展平 -> 全连接 -> 输出

只要前一层的输出形状能作为后一层的输入，这些层就可以灵活搭配。不同 CNN 结构的差别，往往就体现在层的数量、顺序、卷积核个数、是否使用池化等设计上。

## 2.11 关键记忆

- 图像具有局部关联性和一定不变性；CNN 的设计正是为了利用这些图像特征。p3-p6
- 指数衰减下降较快，幂级数衰减下降较慢；幂级数衰减可以看作许多指数衰减成分的叠加。p3-p5
- 类视皮层设计启发 CNN：先提取局部简单特征，再逐步整合成复杂信息。p7-p10
- 卷积的核心是局部窗口加权求和；CNN 相比 MLP 的重要特点是局部连接和参数共享。p11-p13
- Padding 在边缘补值，用来控制输出空间大小；padding 越大，输出空间尺寸通常越大。p14
- Stride 是卷积核滑动步长；stride 越大，输出空间尺寸通常越小。p15
- 多通道卷积中，一个完整卷积核覆盖所有输入通道；一个卷积核产生一个输出通道。p16
- 卷积输出空间尺寸看  $H, W, K, P, S$ ；输出通道数看卷积核个数  $C_{out}$ 。p14-p16
- 卷积参数量看  $K, C_{in}, C_{out}$ ；有 bias 时再加  $C_{out}$ ，不要乘  $H_{out}$  和  $W_{out}$ 。p14-p16
- 池化汇总局部信息，降低空间尺寸，通常没有可学习参数，也通常不改变通道数。p18
- CNN 没有唯一固定格式；卷积、激活、池化、展平、全连接、softmax 等模块可以灵活组合，但相邻层的输入输出尺寸必须匹配。p20-p22

## 2.12 思考题

1. 判断：池化操作一定只能由专门的池化层实现，不能用卷积实现类似效果。

答案：错误。例如使用带 stride 的卷积可以在提取特征的同时降低空间尺寸，起到类似下采样的作用。

- 判断：池化操作可以和卷积操作在同一层的设计中部分合并，例如用 stride 大于 1 的卷积完成下采样。  
答案：正确。
- 填空：1 x 1 卷积核不改变单个位置的空间邻域大小，但可以改变 \_\_\_\_\_ 数。  
答案：通道。
- 计算：输入为 32 x 32 x 3，卷积核为 3 x 3，padding=1，stride=1，卷积核个数为 16，输出尺寸是 \_\_\_\_\_。  
答案：32 x 32 x 16。
- 计算：一个卷积层中 K=5，C\_in=3，C\_out=6，且每个卷积核有 bias，参数量是 \_\_\_\_\_。  
答案：5 x 5 x 3 x 6 + 6 = 456。

### 3 循环神经网络 RNN 与 LSTM

这一章按 PDF 顺序复习：语言任务特点 -> 数据准备 -> 词元化 -> one-hot 与 embedding -> next token prediction -> RNN 设计 -> BPTT -> LSTM。重点是让学生理解：文本是可变长度序列，RNN 为什么适合处理序列，以及 LSTM 的门控机制解决了什么问题。

#### 3.1 自然语言处理 NLP 的任务特点

页码：p3-p5

NLP 的目标是让计算机理解、生成和处理自然语言。常见任务包括机器翻译、文本分类、语音识别、信息检索、智能客服等。

自然语言有层次结构、语法结构、上下文相关性和长程依赖。更重要的是，文本天然是可变长度的离散符号序列。

**自然语言处理 NLP：** NLP 是让计算机理解、生成和处理人类自然语言的技术方向。

文本数据的可变长度：

文本和图像的一个关键区别是：文本序列长度通常不固定。不同句子的 token 数可能不同。

可变长度文本例子：

- 我喜欢 AI
- 我非常喜欢人工智能这门课
- 这部电影虽然节奏很慢，但是后半段非常精彩

这和图像任务很不一样。

**图像与文本输入例子：** 图像通常可以统一 resize 成固定大小，例如 32 x 32 x 3；文本句子却天然长短不一，并且词语顺序会影响含义。

语言数据的几个特点：

| 特点     | 含义              | 例子                |
|--------|-----------------|-------------------|
| 离散符号序列 | 文本由词、子词、字符等符号组成 | “我 / 喜欢 / AI”     |
| 顺序重要   | 词语顺序会影响含义       | “我打他”和“他打我”       |
| 上下文相关  | 当前词含义依赖上下文      | “上海的交通大学”和“上海的交通” |
| 长程依赖   | 较远位置的信息可能影响后面理解 | 前文主语影响后文指代        |

**MLP、CNN 与 RNN 的区别：**

| 模型  | 输入特点      | 是否天然适合可变长度序列 |
|-----|-----------|--------------|
| MLP | 常需要固定长度向量 | 不适合          |
| CNN | 擅长局部窗口特征  | 不天然适合        |

| 模型  | 输入特点         | 是否天然适合可变长度序列 |
|-----|--------------|--------------|
| RNN | 逐 token 处理序列 | 适合           |

RNN 能处理可变长度文本的原因是：它不是一次性要求输入固定长度向量，而是按时间步逐个读入 token，并把已经读过的信息压缩到隐藏状态中。

**RNN 处理可变长度序列的核心：**RNN 按时间步逐个处理输入 token，并在每一步更新隐藏状态；同一个 RNN 单元可以重复使用任意多次，因此可以处理不同长度的序列。

短句：x1 -> x2 -> x3

长句：x1 -> x2 -> x3 -> x4 -> x5 -> ...

同一个 RNN 单元可以在不同时间步重复使用。

## 3.2 数据准备

页码：p6-p7

数据准备是语言模型训练的基础。考试不考复杂数据工程流程，但要知道文本数据进入模型前通常需要清洗和处理。

| 步骤     | 作用                     |
|--------|------------------------|
| 数据收集   | 获得足够数量的文本              |
| 质量过滤   | 去掉质量很差、无意义或噪声很大的文本     |
| 敏感内容过滤 | 减少有毒内容、隐私信息等风险         |
| 数据去重   | 减少重复样本，避免模型过度记忆重复文本    |
| 词元化    | 把文本切成 token            |
| 向量化    | 把 token 变成神经网络能处理的数值向量 |

关键理解：数据不是直接进入 RNN。文本通常要先经过清洗、切分和向量化，才能变成神经网络输入。

## 3.3 分词、one-hot 与 embedding

页码：p8-p11

文本进入神经网络通常要经过两步：先把文本切成 token，再把 token 变成向量。

文本 -> tokenization -> token 序列 -> one-hot 或 embedding 向量

**Tokenization 词元化：**

Tokenization 是把文本切成 token。Token 可以是词、子词、字符或特殊符号。

**Tokenization：**Tokenization 是把原始文本切分成 token 序列的过程；输入是文本，输出是 token 序列。

词元化可以基于规则，也可以基于统计。考试重点不是背具体分词算法，而是知道“文本如何变成 token 序列”。

**Tokenization 例子：**原始句子“我喜欢人工智能”可以切成词级 token：[我, 喜欢, 人工智能]；也可以切成子词级 token：[我, 喜欢, 人工, 智能]。不同 tokenization 方式会产生不同 token 序列。

特殊符号不要求逐个背，但要知道它们也是 token。比如 PAD 常用于把不同长度句子补齐到同一长度，UNK 常用于表示词表中没有出现过的词。

**one-hot 与 embedding：**



文本变成 token 以后，还要进一步变成数值向量。常见表示方式是 one-hot 和 embedding。

**One-hot:** One-hot 是用词表长度的稀疏向量表示一个 token，只有该 token 对应位置为 1，其余位置为 0。

**Embedding:** Embedding 是把 token 映射成低维稠密向量的表示方法，通常由可训练的 embedding 矩阵得到。

| 表示方式      | 特点        | 记忆       |
|-----------|-----------|----------|
| one-hot   | 高维、稀疏     | 不表达词义相似性 |
| embedding | 低维、稠密、可训练 | 可以学习语义关系 |

例子：设词表为“我、喜欢、人工智能、这门课”，token “喜欢”的 one-hot 可以写成  $[0, 1, 0, 0]$ ，维度等于词表大小 4。如果 embedding 维度设为 3，它可能被映射为  $[0.12, -0.38, 0.51]$ 。

Embedding 的计算可以理解为矩阵乘法，也可以理解为查表。设词表大小为  $N$ ，embedding 维度为  $d$ ：

$$\varepsilon_i \in R^N, \quad W_{\text{emb}} \in R^{d \times N}$$

$$x_i = W_{\text{emb}} \varepsilon_i, \quad x_i \in R^d$$

其中， $\varepsilon_i$  是第  $i$  个 token 的 one-hot 向量， $W_{\text{emb}}$  是可训练的 embedding 矩阵， $x_i$  是该 token 的 embedding 向量。

### 3.4 Next Token Prediction

页码：p13-p16

Next Token Prediction 是根据已有前文预测下一个 token。它是现代语言模型常用的训练框架。

**Next Token Prediction:** 给定前文 token 序列，模型预测下一个 token；训练时每个位置的目标通常是输入序列中下一个位置的 token。

## Next Token Prediction 框架



语言模型的训练普遍采用 Next Token Prediction 框架，即输出序列的每一个 token 为输入序列的下一个 token



假设输入为： $X^{\text{in}} = (x_1, x_2, \dots, x_n)$

模型的预测输出为： $\hat{X}^{\text{out}} = (\hat{x}_2, \hat{x}_3, \dots, \hat{x}_{n+1})$

根据 Next Token Prediction，我们期望它的输出为： $X^{\text{out}} = (x_2, x_3, \dots, x_{n+1})$

于是可以对其计算交叉熵损失：

$$\mathcal{L}(X, \hat{X}) = -\frac{1}{n} \sum_{i=2}^{n+1} \sum_{j=1}^d x_{ij} \log \hat{x}_{ij}$$

大量文本都可以转化为“预测下一个 token”的任务：训练时，每个位置预测下一个 token；推理时，模型生成一个 token 后，把它接到上下文后继续生成。图中的  $X^{in}$  表示输入 token 序列，模型输出  $\hat{X}^{out}$ ，目标输出  $X^{out}$  是输入序列整体向后移动一位后的 token 序列。

训练时常用交叉熵损失，真实 token 的概率越高，loss 越小。图中给出的交叉熵损失可以写成：

$$\mathcal{L}(X, \hat{X}) = -\frac{1}{n} \sum_{i=2}^{n+1} \sum_{j=1}^d x_{ij} \log \hat{x}_{ij}$$

其中， $x_{ij}$  表示真实下一个 token 的 one-hot 标签， $\hat{x}_{ij}$  表示模型预测的概率。

### 3.5 RNN 的设计

页码：p17-p18

RNN 是循环神经网络，适合处理序列数据。它的核心设计是：当前隐藏状态不仅看当前输入，也看上一时刻隐藏状态。

## 循环神经网络 (Recurrent Neural Networks)

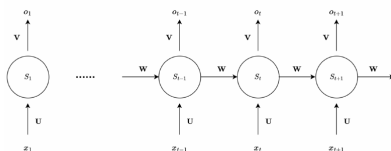


图 3.19: 循环神经网络的基本结构

如图3.19所示，这是一个基本的循环神经网络单元，下面我们详细介绍一下这个单元是如何工作的。我们的输入为时序序列  $x_1, x_2, \dots, x_{t-1}, x_t, x_{t+1}, \dots, x_n$ ，并通过公式(B.15)、(B.16)和(B.17)的方式计算得到对应的输出序列  $o_1, o_2, \dots, o_{t-1}, o_t, o_{t+1}, \dots, o_n$ 。

$$S_0 = 0, \quad (3.15)$$

$$S_t = f(\mathbf{U} \cdot x_t + \mathbf{W} \cdot S_{t-1} + b), \quad t = 1, 2, 3, \dots, n, \quad (3.16)$$

$$o_t = g(\mathbf{V} \cdot S_t) \quad (3.17)$$

其中  $f$  是激活函数，通常使用 Tanh 函数， $g = \text{Softmax}(x)$  是 softmax 函数。 $S_t$  是状态记忆， $\mathbf{U}$ ， $\mathbf{W}$ ， $\mathbf{V}$  和  $b$  分别是输入权重矩阵、循环权重矩阵、输出权重矩阵和偏置项。



**隐藏状态：** RNN 的隐藏状态  $S_t$  是模型在第  $t$  步保存的历史信息摘要，它由当前输入  $x_t$  和上一时刻隐藏状态  $S_{t-1}$  共同决定。

RNN 的隐藏状态可以理解为“目前为止读过内容的压缩记忆”。序列长度不同也没关系，因为 RNN 可以重复使用同一个计算单元，按时间步逐个处理输入。

考试要会识别 RNN 的基本公式：

$$S_0 = 0$$

$$S_t = f(Ux_t + WS_{t-1} + b), \quad t = 1, 2, \dots, n$$

$$o_t = g(VS_t)$$

其中：

设输入维度为  $d_x$ ，隐藏状态维度为  $d_h$ ，输出维度为  $d_y$ ：

| 符号        | 含义             | 形状                   |
|-----------|----------------|----------------------|
| $x_t$     | 第 $t$ 个输入向量    | $R^{d_x}$            |
| $S_t$     | 第 $t$ 步隐藏状态    | $R^{d_h}$            |
| $S_{t-1}$ | 上一时刻隐藏状态       | $R^{d_h}$            |
| $o_t$     | 第 $t$ 步输出      | $R^{d_y}$            |
| $U$       | 输入到隐藏状态的权重矩阵   | $R^{d_h \times d_x}$ |
| $W$       | 隐藏状态到隐藏状态的权重矩阵 | $R^{d_h \times d_h}$ |
| $V$       | 隐藏状态到输出的权重矩阵   | $R^{d_y \times d_h}$ |
| $b$       | 偏置项            | $R^{d_h}$            |

$f$  常用  $\tanh$ ,  $g$  可以是  $\text{softmax}$ 。

这一段的重点只记三件事：

1.  $S_t$  同时依赖当前输入  $x_t$  和历史状态  $S_{t-1}$ 。
2. 参数  $U, W, V$  在不同时间步共享。
3. 输入多少个 token，就重复多少次同样的 RNN 单元，所以可以处理可变长度序列。

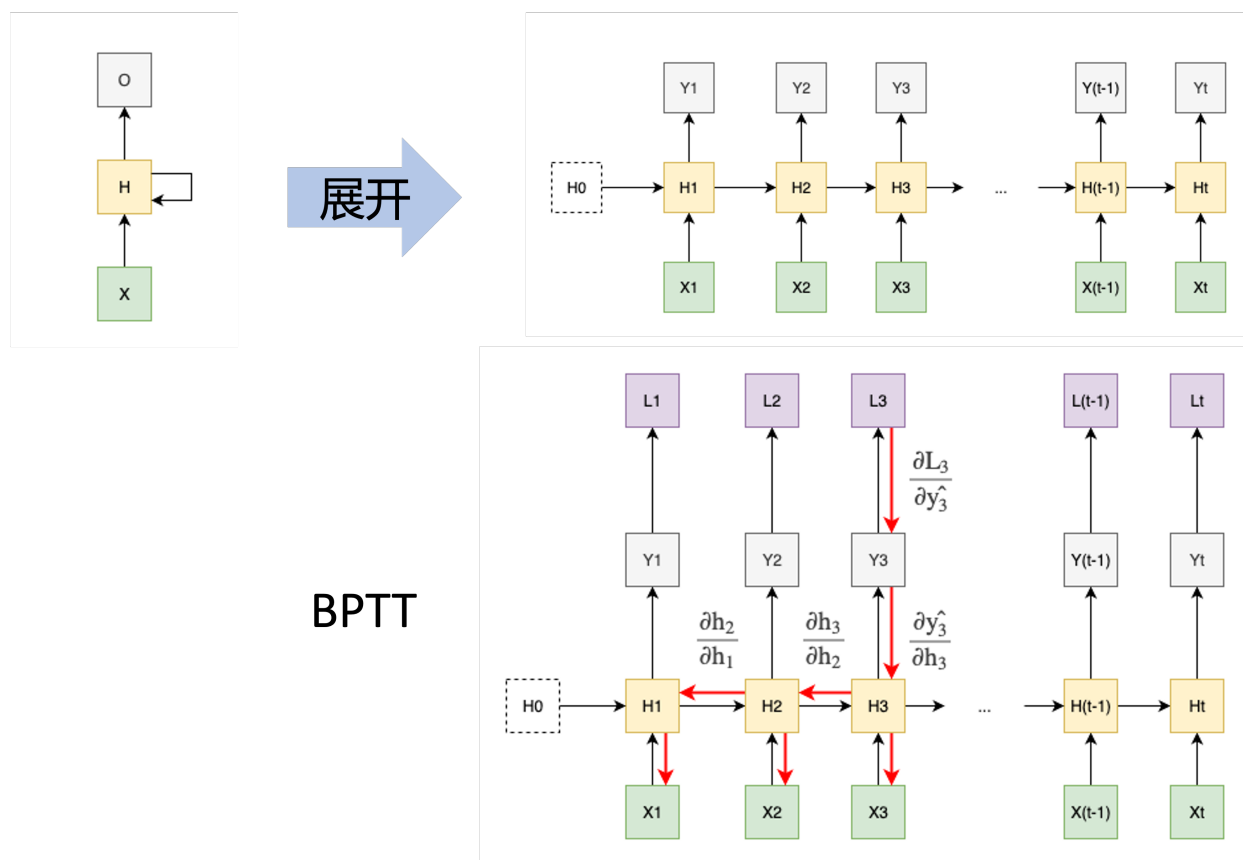
Encoder-Decoder RNN 可以处理输入输出长度不同的任务，例如机器翻译。

### 3.6 BPTT 与梯度问题

页码：p20

BPTT 是 BackPropagation Through Time，意思是随时间反向传播。训练 RNN 时，误差要沿多个时间步向前传播回去。

**BPTT：** BPTT 是随时间反向传播，把 RNN 按时间步展开后，沿时间方向反向计算梯度。



长序列中，梯度需要经过很多次链式相乘，容易出现两个问题：

| 问题   | 含义     | 影响           |
|------|--------|--------------|
| 梯度消失 | 梯度越来越小 | 较早时间步的信息难以学习 |
| 梯度爆炸 | 梯度越来越大 | 训练不稳定        |

因此，普通 RNN 对长程依赖的建模能力有限。

如果序列非常长，理论上完整 BPTT 要展开并回传很多时间步，计算和显存开销都很大。实际训练中常用截断 BPTT，只向前回看有限步数，而不是对无限长历史完整反传。

### 3.7 LSTM

页码: p21-p25

LSTM 是 Long Short-Term Memory, 长短期记忆网络。它在 RNN 基础上引入记忆状态, 并通过门控机制控制信息流动。

## 长短期记忆网络 (Long short-term memory, LSTM)

- 基本原则: 1) 任何信息使用前先做线性变换, 权重可学  
2) 控制比例的称为“门”, 激活用 Sigmoid  
3) 提取信息的运算, 激活用 Tanh

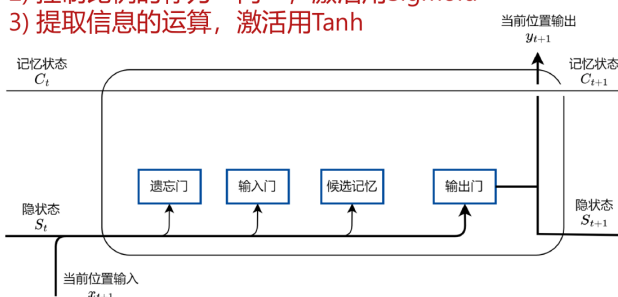


图 3.21: 单个门控神经元主要部件和并行隐藏状态示意图

为什么能称为“遗忘门”、“输入门”、“输出门”、“知识”？  
这是一种形象类比, 没有严格证明。

#### 门控神经元-遗忘门

保留的比例  $f_t = \sigma(W_{Sf}S_t + W_{xf}x_{t+1} + b_f)$

#### 门控神经元-输入门

写入的比例  $i_t = \sigma(W_{Si}S_t + W_{xi}x_{t+1} + b_i)$

新的知识  $\tilde{C}_{t+1} = \tanh(W_{SC}S_t + W_{xC}x_{t+1} + b_C)$

新的记忆状态  $C_{t+1} = f_t * C_t + i_t * \tilde{C}_{t+1}$

#### 门控神经元-输出门

输出的比例  $o_t = \sigma(W_{So}S_t + W_{xo}x_{t+1} + b_o)$

输出  $y_{t+1} = S_{t+1} = o_t * \tanh(C_{t+1})$



**LSTM:** LSTM 是一种改进的 RNN, 它通过记忆状态和门控机制控制信息的保留、写入和输出, 用来缓解普通 RNN 的长程依赖问题。

| 结构   | 作用         |
|------|------------|
| 记忆状态 | 保存较长期的信息   |
| 遗忘门  | 控制保留多少旧记忆  |
| 输入门  | 控制写入多少新信息  |
| 输出门  | 控制输出多少记忆信息 |

门通常使用 sigmoid, 因为 sigmoid 输出在 0 到 1 之间, 适合表示“保留多少”“写入多少”这种比例。候选记忆常用 tanh 提取。

**门控机制:** 门控机制用 0 到 1 之间的比例控制信息流动: 遗忘门控制旧记忆保留多少, 输入门控制新信息写入多少, 输出门控制当前输出多少记忆信息。

LSTM 设计里最重要的点:

1. 多了一条记忆状态  $C_t$ , 用于保存较长期信息。
2. 用门控制比例, 而不是简单地全保留或全丢弃。
3. 遗忘门决定旧记忆保留多少。
4. 输入门决定新信息写入多少。
5. 输出门决定当前输出多少记忆信息。

普通 RNN 与 LSTM 的区别可以概括为：

| 对比     | 普通 RNN           | LSTM                   |
|--------|------------------|------------------------|
| 保存历史信息 | 主要依靠隐藏状态 $S_t$   | 额外引入记忆状态 $C_t$         |
| 信息控制   | 缺少显式门控           | 用遗忘门、输入门、输出门控制信息比例     |
| 长程依赖   | 长序列中更容易梯度消失或梯度爆炸 | 通过记忆状态和门控机制缓解长程依赖      |
| 很长序列训练 | 常配合截断 BPTT       | 仍可配合截断 BPTT，但更容易保留长期信息 |

截断 BPTT 解决的是“序列太长，不能无限展开反传”的训练开销问题；LSTM 解决的是“普通 RNN 难以保留长期信息”的建模问题。LSTM 不能保证完全消除所有梯度问题，但它可以缓解普通 RNN 的长程依赖和梯度消失问题。

### 3.8 关键记忆

- NLP 处理的是离散符号序列，词语顺序和上下文都很重要。
- 文本数据通常是可变长度的；MLP/CNN 不天然适合记忆历史信息，RNN 通过隐藏状态逐步处理序列。
- 文本进入神经网络前通常要经过数据清洗、tokenization 和向量化。
- Tokenization 是把文本切成 token；token 可以是词、子词、字符或特殊符号。
- One-hot 高维稀疏，本身不表达词义相似性；embedding 低维稠密，可以通过训练学习语义关系。
- Next Token Prediction 是根据前文预测下一个 token，是现代语言模型常用训练框架。
- RNN 的当前隐藏状态依赖当前输入和上一时刻隐藏状态，公式为  $S_t = f(Ux_t + WS_{t-1} + b)$ 。
- BPTT 是随时间反向传播；长序列中容易出现梯度消失或梯度爆炸。
- LSTM 通过记忆状态和门控机制控制信息保留、写入和输出，可以缓解长程依赖问题。

### 3.9 思考题

1. 判断：文本数据通常是可变长度序列，因此普通 MLP 不天然适合直接处理任意长度文本。  
答案：正确。
2. 填空：把文本切成 token 的过程叫做 \_\_\_\_\_。  
答案：tokenization 或词元化。
3. 填空：RNN 的基本公式中，当前隐藏状态可以写成  $S_t = f(Ux_t + W \_\_\_\_\_\_ + b)$ 。  
答案： $S_{t-1}$ 。
4. 判断：RNN 可以处理可变长度序列，一个重要原因是同一个 RNN 单元可以在不同时间步重复使用。  
答案：正确。
5. 选择：LSTM 中控制保留多少旧记忆的是：A. 输入门 B. 遗忘门 C. 输出门 D. Softmax。  
答案：B。

## 4 Transformer

这一章按 PDF 顺序复习：为什么需要 Transformer -> next token prediction -> embedding 与位置编码 -> 注意力机制与 Q/K/V -> causal mask -> Transformer block -> 输出层。重点是理解 Transformer 如何用注意力机制建模 token 之间的关系，以及为什么它比 RNN 更适合并行计算。

### 4.1 为什么需要 Transformer

页码：p3-p9

Transformer 出现前，序列任务常用 RNN/LSTM。RNN/LSTM 能处理序列，但有两个明显限制：长程依赖不容易保留，并且计算通常要按时间步顺序进行，不方便并行。

# Attention is all you need!



## Why Transformer?

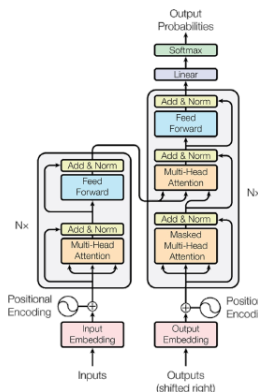
Transformer<sup>[1]</sup> 诞生之前，在自然语言处理中大都采用基于RNN的编码器-解码器（Encoder-Decoder）结构来完成序列翻译。

### RNN (LSTM) 的不足：

- 难以捕获长期依赖关系（梯度消失和梯度爆炸仍存在）

### Transformer的优势：

- 更好地捕捉全局语义关系（多头注意力机制）
- 更好地理解上下文（位置编码）
- 更快的计算效率！（结构使得高效的并行计算成为可能）



[1] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[J]. Advances in neural information processing systems, 2017, 30.



**Transformer：** Transformer 是以注意力机制为核心的序列模型，能够直接建模 token 之间的关系，更容易捕捉全局上下文，也更适合并行计算。

Transformer 的直观思想可以理解为：当前 token 更新自己时，不是只看前一个隐藏状态，而是主动计算“我应该关注哪些 token”，再按关注程度汇总信息。

| 模型          | 处理序列的方式          | 主要特点              |
|-------------|------------------|-------------------|
| RNN/LSTM    | 按时间步顺序读入 token   | 有隐藏状态，但并行不方便      |
| Transformer | 直接计算 token 之间的关系 | 注意力机制更适合全局关系和并行计算 |

**例子 1：**注意力的直观类比。如果要估计某位同学缺失的化学成绩，可以先找“和他各科表现相似”的同学，再参考这些同学的化学成绩。注意力机制做的事情也类似：先计算相关性，再按相关性汇总信息。

## 4.2 Next Token Prediction 框架

页码：p10-p13

Transformer 语言模型常用于 next token prediction：给定前文 token 序列，预测下一个 token。训练时，输入序列和目标序列通常相差一个位置。

**Next Token Prediction：** 给定前文 token，模型预测下一个 token；训练时每个位置的目标通常是下一个位置的真实 token。

输入和输出：

设序列长度为  $n$ ，词表大小为  $d$ ，one-hot 表示后的输入和目标可以写成：

$$X^{in} \in R^{n \times d}, \quad X^{out} \in R^{n \times d}$$

如果输入是“我 爱 小 猫”，目标可以理解为整体向后移动一位：

例子 2: 输入和目标错位。

| 输入位置 | 目标 token |
|------|----------|
| 我    | 爱        |
| 爱    | 小        |
| 小    | 猫        |
| 猫    | !        |

交叉熵损失:

如果  $x_{ij}$  是真实下一个 token 的 one-hot 标签,  $\hat{x}_{ij}$  是模型预测概率, 则常用交叉熵损失:

$$\mathcal{L}(X, \hat{X}) = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^d x_{ij} \log \hat{x}_{ij}$$

真实 token 的预测概率越高, loss 越小。

### 4.3 Embedding 与位置编码

页码: p15-p18

Token 是离散符号, 不能直接进入神经网络。Embedding 层把 one-hot token 映射成低维稠密向量。

#### 嵌入 Embedding



$$X^{\text{in}} \in \mathbb{R}^{n \times d}$$

$$X^{\text{em}} = X^{\text{in}} W^{\text{em}} \in \mathbb{R}^{n \times d_m}$$

$$W^{\text{em}} \in \mathbb{R}^{d \times d_m} \quad \text{随机初始化, 可训练}$$

$x_i \in \mathbb{R}^d$  表示第  $i$  个词的  $d$  维 one-hot 表征向量  $d$  为字典大小

“我”:  $[\dots, 1, \dots, 0, \dots, 0, \dots, 0, \dots]$  (第100 位 = 1)

“爱”:  $[\dots, 0, \dots, 1, \dots, 0, \dots, 0, \dots]$  (第205 位 = 1)

“小”:  $[\dots, 0, \dots, 0, \dots, 1, \dots, 0, \dots]$  (第310 位 = 1)

“猫”:  $[\dots, 0, \dots, 0, \dots, 0, \dots, 1, \dots]$  (第402 位 = 1)



**Embedding:** Embedding 是把 token 映射成低维稠密向量的表示方法, 通常由可训练的 embedding 矩阵得到。

**Embedding 公式:**

设词表大小为  $d$ , embedding 维度为  $d_m$ :

$$X^{\text{in}} \in \mathbb{R}^{n \times d}, \quad W^{\text{emb}} \in \mathbb{R}^{d \times d_m}$$

$$X^{\text{emb}} = X^{\text{in}} W^{\text{emb}}, \quad X^{\text{emb}} \in \mathbb{R}^{n \times d_m}$$



例子 3: **embedding** 矩阵大小。如果词表大小  $d = 10000$ , embedding 维度  $d_m = 128$ , 则 embedding 矩阵大小为  $10000 \times 128$ 。

只有 embedding 还不够, 因为注意力机制本身不天然知道 token 的顺序。位置编码用于告诉模型每个 token 在序列中的位置。

## 嵌入 Embedding + 位置编码



$$X^{\text{in}} \in \mathbb{R}^{n \times d}$$

$$X^{\text{em}} = X^{\text{in}} W^{\text{em}} \in \mathbb{R}^{n \times d_m}$$

$$W^{\text{em}} \in \mathbb{R}^{d \times d_m} \quad \text{随机初始化, 可训练}$$

$$X^{\text{pos}} \in \mathbb{R}^{n \times d_m}$$

$$X^{(1)} = X^{\text{em}} + X^{\text{pos}}$$

$X^{\text{pos}}$ : 可以是可学的参数, 也可以是针对绝对位置或者相对位置的不可学习量。



位置编码: 位置编码为模型提供 token 的位置信息; 输入表示通常由 token embedding 和位置编码相加得到。

输入表示:

$$X^{(1)} = X^{\text{emb}} + X^{\text{pos}}$$

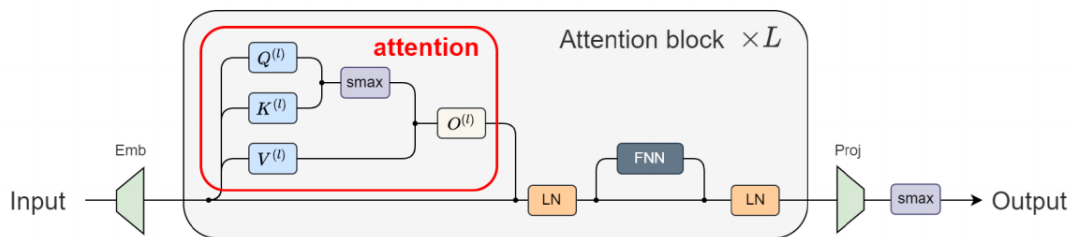
其中,  $X^{\text{pos}}$  可以是可学习参数, 也可以是人为设计的位置编码。课件后面的复杂位置编码属于阅读材料, 复习时不展开推导。

## 4.4 注意力机制与 Q、K、V

页码: p21-p31

注意力机制可以分成四步: 先把输入线性变换成 Q、K、V; 再用 Q 和 K 算关联强度; 然后经过 mask 和 softmax 得到注意力权重; 最后用这些权重对 V 加权求和。

## 注意力机制



$$\begin{aligned} Q^{(l)} &= X^{(l)} W^{Q^{(l)}}, W^{Q^{(l)}} \in R^{d_m \times d_k} \\ K^{(l)} &= X^{(l)} W^{K^{(l)}}, W^{K^{(l)}} \in R^{d_m \times d_k} \\ V^{(l)} &= X^{(l)} W^{V^{(l)}}, W^{V^{(l)}} \in R^{d_m \times d_v} \end{aligned}$$

$l$ 表示层的指标



**Q、K、V**：Q 是 query，用来发出查询；K 是 key，用来和 query 匹配；V 是 value，是真正被加权汇总的信息。

设第  $l$  层输入为  $X^{(l)} \in R^{n \times d_m}$ ：

$$Q^{(l)} = X^{(l)} W^{Q^{(l)}}, \quad W^{Q^{(l)}} \in R^{d_m \times d_k}$$

$$K^{(l)} = X^{(l)} W^{K^{(l)}}, \quad W^{K^{(l)}} \in R^{d_m \times d_k}$$

$$V^{(l)} = X^{(l)} W^{V^{(l)}}, \quad W^{V^{(l)}} \in R^{d_m \times d_v}$$

因此：

$$Q^{(l)}, K^{(l)} \in R^{n \times d_k}, \quad V^{(l)} \in R^{n \times d_v}$$

注意力权重：

$$A^{(l)} = \frac{Q^{(l)} (K^{(l)})^T}{\sqrt{d_k}}$$

$$\text{Attn}^{(l)} = \text{softmax}(\text{mask}(A^{(l)}))$$

其中，除以  $\sqrt{d_k}$  主要是为了训练更稳定。

加权求和：

$$X^{qkv(l)} = \text{Attn}^{(l)} V^{(l)}, \quad X^{qkv(l)} \in R^{n \times d_v}$$

**例子 4**：注意力加权求和。如果某个 token 对三个位置的注意力权重是  $[0.7, 0.2, 0.1]$ ，三个 value 分别是  $[1, 3, 5]$ ，则汇总结果为

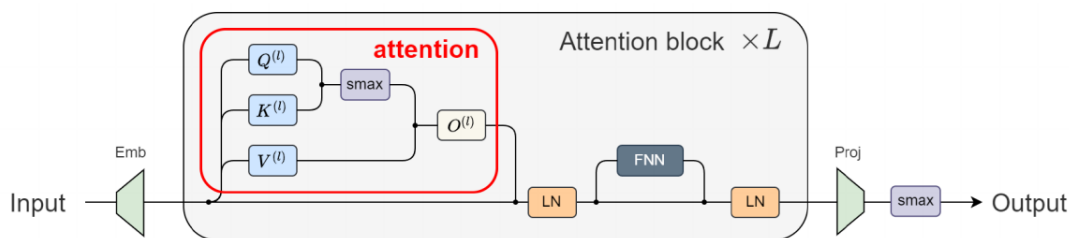
$$0.7 \times 1 + 0.2 \times 3 + 0.1 \times 5 = 1.8$$

## 4.5 Causal Mask

页码: p27-p29

在生成任务中, 预测当前位置时不能看到未来 token, 否则模型就相当于提前看到了答案。Causal mask 用来遮住未来位置。

### 注意力机制



### 算关联+Mask+归一化

除  $\sqrt{d_k}$  是为训练稳定

$$\text{Attn}^{(l)} = \text{softmax} \left( \frac{\text{mask}(Q^{(l)}(K^{(l)})^T)}{\sqrt{d_k}} \right) \in \mathbb{R}^{n \times n}$$

$$\text{softmax}(A)_{ij} = \frac{e^{a_{ij}}}{\sum_{j=1}^m e^{a_{ij}}}$$

$$\begin{bmatrix} 0 & 0 & \cdots & \cdots & 0 \\ & 0 & \cdots & 0 & 0 \\ & & 0 & \cdots & \cdots \\ & & & 0 & 0 \\ & & & & 0 \end{bmatrix}$$

元素  $ij$ : 第  $i$  个词对第  $j$  个词的注意值  
只能对自己前面的词有关!



**Causal Mask:** Causal mask 会把未来位置的注意力分数设为很小的值, 使 softmax 后这些位置的权重接近 0, 从而保证预测时不能看未来 token。

**例子 5:** 不能偷看未来。已知前文是“我 爱”, 模型要预测下一个 token 时, 可以看“我”和“爱”, 但不能提前看后面的“小 猫”。

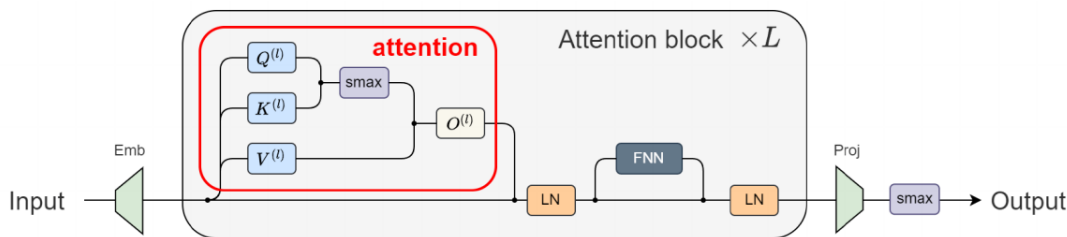
对第  $i$  个 token 来说, causal mask 只允许它关注第 1 到第  $i$  个 token。这样就保证 next token prediction 的因果性。

## 4.6 Transformer Block、LayerNorm 与 FNN

页码: p31-p37

注意力输出后, 还会经过线性变换、残差连接和 LayerNorm。之后再接前馈全连接网络 FNN。多个 Transformer block 可以复制堆叠。

## 注意力机制



### 算关联+Mask+归一化+加权求和+线性变换

$$X^{pr(l)} = X^{qkv(l)} W^{O(l)} \in \mathbb{R}^{n \times d_m}, \quad W^{O(l)} \in \mathbb{R}^{d_v \times d_m}$$

$$X^{ao(l)} = \text{LayerNorm}(X^{(l)} + X^{pr(l)}) \in \mathbb{R}^{n \times d_m}$$

$$[\text{LayerNorm}(X)]_{i,j} = \gamma * \frac{X_{i,j} - \sum_l X_{i,l}/d_m}{\text{std}(X_{i,1}, \dots, X_{i,d_m})} + \beta. \quad \gamma \text{ 和 } \beta \text{ 可训练}$$



**Transformer Block:** 一个 Transformer block 通常包含注意力模块、残差连接、LayerNorm 和前馈全连接网络 FNN。

注意力输出后的线性变换:

$$X^{pr(l)} = X^{qkv(l)} W^{O(l)}, \quad W^{O(l)} \in \mathbb{R}^{d_v \times d_m}$$

残差连接与 LayerNorm:

$$X^{ao(l)} = \text{LayerNorm}(X^{(l)} + X^{pr(l)})$$

LayerNorm 是逐个 token 做归一化: 每个 token 自己减均值、除方差, 再做可训练的缩放和平移。复习时只需知道它用于稳定训练, 不需要展开复杂变体。

**FNN:**

FNN 是前馈全连接网络, 对每个 token 分别作用。它通常不混合不同 token 的信息, token 之间的信息交互主要来自注意力模块。

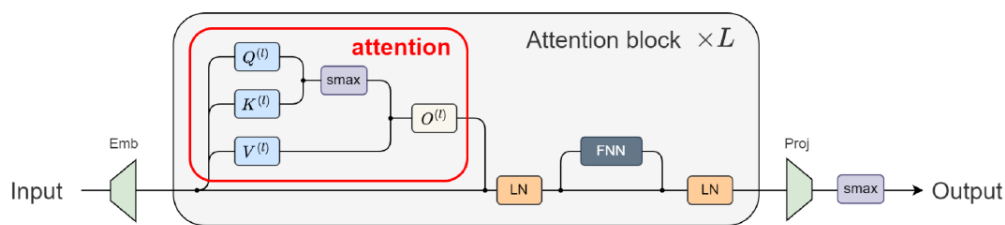
**例子 6:** 逐 token 过 FNN。对句子“我爱小猫”, FNN 会分别作用在“我”“爱”“小”“猫”对应的隐藏向量上; 不同 token 之间的信息交换主要已经在注意力模块中完成。

## 4.7 输出投影与解码

页码: p38-p39

最后, 输出投影层把每个 token 的隐藏向量映射回词表大小, 再经过 softmax 得到下一个 token 的概率分布。

## 输出 (投影) 层



$$X^{\text{out}} = X^{\text{do}(L)} W^{\text{proj}} + b^{\text{proj}} \in \mathbb{R}^{n \times d}, \quad W^{\text{proj}} \in \mathbb{R}^{d_m \times d}, \quad b^{\text{proj}} \in \mathbb{R}^d$$

贪婪解码 (greedy decoding)  $\text{Output} = \text{argmax}(\text{softmax}(X^{\text{out}}))$

采样解码 (sampling decoding)  $\text{Output} \sim \text{softmax}(X^{\text{out}})$



输出投影层：输出投影层把隐藏向量映射到词表维度，softmax 后得到每个候选 token 的概率。

输出公式：

$$X^{\text{out}} = X^{\text{do}(L)} W^{\text{proj}} + b^{\text{proj}}, \quad W^{\text{proj}} \in \mathbb{R}^{d_m \times d}$$

其中， $d$  是词表大小，所以输出的最后一维对应词表中每个 token 的分数。

| 解码方式 | 含义              | 结果是否固定 |
|------|-----------------|--------|
| 贪婪解码 | 每一步取概率最大的 token | 通常固定   |
| 采样解码 | 按概率分布随机抽样       | 可能不同   |

例子 7：贪婪解码。如果词表中“猫”的概率是 0.70，“狗”的概率是 0.20，“鼠”的概率是 0.10，贪婪解码会选择“猫”。

## 4.8 关键记忆

- Transformer 用注意力机制直接建模 token 之间的关系，比 RNN 更适合并行计算。p3-p9
- Next token prediction 是根据前文预测下一个 token，常用交叉熵损失。p10-p13
- Embedding 把 one-hot token 映射成低维稠密向量；位置编码提供 token 位置信息。p15-p18
- Q 和 K 用于计算关联强度，V 是真正被加权汇总的信息。p21-p26
- 注意力的核心计算是  $QK^T$ 、mask、softmax、再对 V 加权求和。p26-p31
- Causal mask 保证预测当前位置时不能看到未来 token。p27-p29
- Transformer block 通常包含注意力模块、残差连接、LayerNorm 和 FNN。p31-p37
- 输出投影层把隐藏向量映射到词表维度；贪婪解码取最大概率 token，采样解码具有随机性。p38-p39

## 4.9 思考题

- 判断：Transformer 使用注意力机制来建模 token 之间的关系。  
答案：正确。

2. 填空：Transformer 需要加入 \_\_\_\_\_，用来表示 token 在序列中的位置。  
答案：位置编码。
3. 选择：Q、K、V 中，用来真正汇总信息内容的是：A. Q B. K C. V D. mask。  
答案：C。
4. 判断：Decoder 做 next token prediction 时，可以看到未来 token。  
答案：错误。需要 causal mask 遮住未来位置。
5. 计算：注意力权重为  $[0.6, 0.4]$ ，两个 value 为  $[2, 5]$ ，加权求和结果是多少？  
答案： $0.6 \times 2 + 0.4 \times 5 = 3.2$ 。

## 5 神经网络训练现象：频率原则

这一章不适合考复杂推导，重点考概念和直觉。

### 5.1 过参数化与隐式偏好

复习重点：

1. 过参数化指模型参数量大于样本量或明显很大。
2. 过参数化模型理论上容易表达很多复杂函数。
3. 但神经网络实际训练中常常仍能泛化较好。
4. 训练误差为 0 的解可能有很多。
5. 神经网络训练过程会倾向某些解，这叫隐式偏好。

易错点：

1. “过参数化神经网络一定严重过拟合。”错误。
2. “训练过程可能偏向某类解。”正确。
3. “隐式偏好一定写在损失函数公式里。”错误。

### 5.2 频率、低频与高频

复习重点：

1. 频率描述函数变化的快慢。
2. 低频对应平滑、轮廓、整体趋势。
3. 高频对应振荡、细节、快速变化。
4. 图像中低频常对应大轮廓，高频常对应边缘、纹理或噪声。
5. 高频不一定没用，但更容易包含噪声。

易错点：

1. “低频通常更平滑。”正确。
2. “高频永远没有用。”错误。
3. “细节和噪声都可能表现为高频。”正确。

### 5.3 频率原则

复习重点：

1. 频率原则也叫 spectral bias。
2. 神经网络训练通常先拟合低频，再拟合高频。
3. 在一维函数中表现为先学平滑轮廓，再学振荡细节。
4. 在图像拟合中表现为先学整体轮廓，再学局部细节。
5. 频率原则可以帮助解释为什么一些神经网络不容易立即拟合噪声。

必背句：

频率原则：神经网络在训练过程中通常按照从低频到高频的顺序拟合目标函数。

易错点：

1. “频率原则说神经网络通常先学高频噪声。”错误。
2. “频率原则和泛化有关系。”正确。
3. “频率原则只适用于分类任务，不能用于函数拟合直觉。”错误。

## 5.4 早停 Early stopping

复习重点：

1. 数据中可能有噪声。
2. 低频成分通常先被模型学习。
3. 高频噪声往往在训练后期被逐渐拟合。
4. 早停是在模型过度拟合噪声前停止训练。
5. 早停有助于提高测试集泛化能力。

易错点：

1. “训练越久，测试误差一定越低。”错误。
2. “早停可能防止拟合高频噪声。”正确。
3. “早停就是让模型完全不训练。”错误。

## 5.5 思考题

1. 填空：频率原则也叫 spectral \_\_\_\_\_。  
答案：bias。
2. 判断：神经网络训练通常先拟合低频成分，再拟合高频成分。  
答案：正确。
3. 选择：图像中的边缘、纹理、噪声通常更接近：A. 低频 B. 高频 C. 标签 D. bias。  
答案：B。
4. 判断：过参数化神经网络一定会严重过拟合，因此不可能泛化好。  
答案：错误。
5. 判断：早停可以理解为在模型进一步拟合高频噪声前停止训练。  
答案：正确。

# 6 参数凝聚与解的平坦性

## 6.1 初始化大小的影响

复习重点：

1. 初始化是训练开始前给参数赋初值。
2. 初始化会影响训练过程和最终结果。
3. 初始化太大时，网络输出可能更随机、更复杂，更容易过拟合。
4. 初始化较小时，网络输出可能更平滑。
5. 常见初始化包括 LeCun、Xavier、Kaiming。
6. 模型越大时，初始化尺度通常需要更谨慎。
7. 所有参数不能简单初始化为 0，因为会破坏训练中的对称性打破。

易错点：

1. “初始化对训练结果没有任何影响。”错误。
2. “所有参数初始化为 0 是最安全的做法。”错误。
3. “初始化大小会影响训练。”正确。

## 6.2 参数凝聚

复习重点：

1. 参数凝聚指小初始化下，同层神经元会有趋同的偏好。
2. 多个相似神经元可以等效为较少的有效神经元。

3. 凝聚使大网络在表达上像复杂度更低的小网络。
4. 过参数化大网络发生凝聚后，仍然比直接训练小网络更容易优化。
5. 凝聚可理解为一种复杂度控制。

必背句：

参数凝聚：小初始化下，同层神经元会出现趋同的偏好，多个神经元可能等效为较少的有效神经元。

易错点：

1. “参数凝聚通常与小初始化有关。”正确。
2. “凝聚意味着网络完全不能训练。”错误。
3. “凝聚可以看作降低有效复杂度的一种现象。”正确。

## 6.3 平坦解与尖锐解

复习重点：

1. 平坦解指参数附近一片区域损失都较低。
2. 尖锐解指参数稍微改变，损失就明显上升。
3. 训练集和测试集常有噪声，平坦解对扰动更鲁棒。
4. 平坦解通常泛化更好。
5. 小批量训练倾向于找到更平坦的解。
6. 合理较大的学习率也可能帮助找到更平坦的解。
7. Dropout 也可能有助于提升解的平坦性。

易错点：

1. “平坦解对参数扰动更鲁棒。”正确。
2. “尖锐解一定比平坦解泛化更好。”错误。
3. “小批量训练可能帮助找到更平坦的解。”正确。

## 6.4 思考题

1. 判断：初始化大小会影响神经网络的训练过程和最终结果。  
答案：正确。
2. 判断：所有参数初始化为 0 通常不是好做法，因为会破坏对称性打破。  
答案：正确。
3. 填空：小初始化下，同层神经元可能出现趋同偏好，这种现象叫 \_\_\_\_\_。  
答案：参数凝聚。
4. 选择：平坦解通常指：A. 参数附近一片区域损失都较低 B. 参数必须全为 0 C. 训练误差一定为 1 D. 没有梯度。  
答案：A。
5. 判断：尖锐解对参数扰动更敏感，泛化通常不如平坦解稳健。  
答案：正确。

# 7 大模型介绍

## 7.1 什么是大模型

复习重点：

1. 大模型通常也叫基础模型。
2. 大模型参数量很大。
3. 大模型经过大规模数据训练。
4. 大模型可以适应多种下游任务。
5. 大语言模型是最早展现出通用智能特征的大模型之一。
6. 许多模态的数据也可以类似语言一样进行 token 化。

易错点：



1. “大模型通常经过大规模数据训练。”正确。
2. “大模型只能做一个固定任务。”错误。
3. “基础模型可以适应多种下游任务。”正确。

## 7.2 Encoder、Decoder 与常见架构

复习重点：

1. Encoder 可以看到上下文所有 token。
2. Decoder 只能看到当前位置和前文 token。
3. Encoder-only 常用于理解任务。
4. Decoder-only 常用于生成任务。
5. Encoder-Decoder 常用于输入和输出都需要建模的任务，如翻译。
6. BERT 是 Encoder-only 的代表。
7. GPT、Llama、DeepSeek 等是 Decoder-only 的代表。
8. BART、T5 是 Encoder-Decoder 的代表。
9. 当前主流生成式大语言模型多采用 Decoder-only。

对照表：

| 架构              | 是否看未来     | 常见任务 | 代表                 |
|-----------------|-----------|------|--------------------|
| Encoder-only    | 可以看双向上下文  | 理解任务 | BERT               |
| Decoder-only    | 只能看当前和前文  | 生成任务 | GPT/Llama/DeepSeek |
| Encoder-Decoder | 编码输入，解码输出 | 翻译等  | T5/BART            |

易错点：

1. “Decoder-only 在生成任务中很常见。”正确。
2. “Encoder-only 非常适合直接做 next token prediction。”通常不适合。
3. “BERT 是 Decoder-only。”错误。

## 7.3 预训练、微调、提示词、蒸馏

复习重点：

1. 预训练是在大规模数据上训练基础能力。
2. 语言模型预训练常用 next token prediction。
3. 微调是在预训练模型基础上，用特定任务数据继续训练。
4. 强化学习后训练可增强推理能力，也可用于对齐人类偏好。
5. 提示词工程通过设计输入让模型给出更好的回答。
6. 知识蒸馏把大模型能力迁移到小模型中。
7. Teacher model 是提供知识的大模型或强模型。
8. Student model 是接受蒸馏的小模型或目标模型。

易错点：

1. “预训练通常需要大量数据。”正确。
2. “提示词工程一定会更新模型参数。”错误。
3. “知识蒸馏可以让小模型学习大模型输出中的知识。”正确。

## 7.4 大模型中的现象

复习重点：

1. Scaling law 描述模型规模、数据量、计算量和测试损失之间的规律关系。
2. 一般来说，在一定范围内，规模、数据、算力增加会带来性能提升。
3. 上下文学习指模型不更新参数，仅通过任务描述和示例适应任务。
4. 思维链是让模型生成中间推理步骤。

5. 思维链可提升复杂推理任务表现。
6. 幻觉是模型生成看似真实但实际错误或误导的信息。
7. 大模型可能包含偏见。

易错点：

1. “上下文学习不需要更新模型权重。”正确。
2. “思维链就是引导模型写出中间推理步骤。”正确。
3. “大模型永远不会产生幻觉。”错误。
4. “大模型可能带有偏见。”正确。

## 7.5 思考题

1. 判断：大模型通常经过大规模数据训练，可以适应多种下游任务。  
答案：正确。
2. 选择：当前主流生成式大语言模型多采用：A. Encoder-only B. Decoder-only C. 纯 CNN D. 纯 RNN。  
答案：B。
3. 填空：预训练是在大规模数据上训练模型的 \_\_\_\_\_ 能力。  
答案：基础。
4. 判断：上下文学习通常不更新模型参数，而是通过任务描述和示例让模型适应任务。  
答案：正确。
5. 判断：幻觉指模型生成看似真实但实际错误或误导的信息。  
答案：正确。

## 8 强化学习

这一章是考试重点，但不建议考复杂推导。重点是概念、关系和最简单公式。

### 8.1 强化学习是什么

复习重点：

1. 强化学习是决策型任务。
2. 智能体在环境中采取动作。
3. 环境返回奖励和新的状态。
4. 智能体通过反复试错学习。
5. 目标是最大化期望累计奖励。
6. 奖励可以是正的，也可以是负的。
7. 强化学习不同于监督学习：通常没有每一步的正确标签，而是通过奖励反馈学习。

基本循环：

状态  $s_t$   $\rightarrow$  智能体选择动作  $a_t$   $\rightarrow$  环境给出奖励  $r_{t+1}$  和下一状态  $s_{t+1}$

易错点：

1. “强化学习只做静态分类任务。”错误。
2. “强化学习强调智能体和环境交互。”正确。
3. “奖励只能是正数。”错误。

### 8.2 状态、观察、动作空间、奖励

复习重点：

1. 状态 State 是环境完整信息。
2. 观察 Observation 是智能体实际看到的信息。
3. 完全可观测环境中，观察和状态一致。
4. 部分可观测环境中，观察只是状态的一部分。
5. 动作空间是所有可选动作的集合。

6. 动作空间可以是离散的，也可以是连续的。
7. 棋类、超级马里奥常可看作离散动作空间。
8. 自动驾驶中的方向盘角度、油门等常可看作连续动作空间。
9. 奖励是评价动作好坏的反馈信号。

易错点：

1. “观察一定等于完整状态。”错误。
2. “动作空间可以是离散的，也可以是连续的。”正确。
3. “奖励用于告诉智能体行为好坏。”正确。

### 8.3 回报与折扣因子

复习重点：

1. 强化学习不只关心当前奖励，还关心未来奖励。
2. 回报是未来奖励的累积。
3. 折扣因子  $\gamma$  用于降低远期奖励权重。
4.  $\gamma$  的取值范围是  $0 \leq \gamma \leq 1$ 。
5.  $\gamma$  越接近 0，越重视眼前奖励。
6.  $\gamma$  越接近 1，越重视长期奖励。

无折扣回报：

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

折扣回报：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

例题：未来三个奖励分别为 1, 0, 2,  $\gamma = 0.5$ 。

$$G_t = 1 + 0.5 \times 0 + 0.5^2 \times 2 = 1.5$$

易错点：

1. “强化学习完全不关心未来奖励。”错误。
2. “折扣因子通常在 0 到 1 之间。”正确。
3. “ $\gamma$  越接近 1，越重视长期奖励。”正确。

### 8.4 情节型任务与连续型任务

复习重点：

1. 情节型任务有起点和终点。
2. 一局游戏、一次棋局、一个关卡通常是情节型任务。
3. 连续型任务没有固定终止状态。
4. 股票交易、交通控制、自动驾驶可看作连续型任务。

易错点：

1. “所有强化学习任务都有明确终点。”错误。
2. “棋类游戏通常可以看作情节型任务。”正确。
3. “连续型任务需要持续决策。”正确。

### 8.5 探索与利用

复习重点：

1. 探索 Exploration 是尝试未知或不确定动作以获取信息。
2. 利用 Exploitation 是根据已有知识选择当前看起来最好的动作。

3. 只利用可能错过更好的策略。
4. 只探索可能无法获得足够高奖励。
5. 强化学习需要平衡探索和利用。

生活例子：每天去熟悉但还不错的餐厅是利用；尝试一家没去过的新餐厅是探索。

易错点：

1. “探索是尝试未知动作。”正确。
2. “利用是根据已有知识做当前较优选择。”正确。
3. “强化学习中完全不需要探索。”错误。

## 8.6 策略、价值函数与最优策略

复习重点：

1. 策略 policy 是智能体选择动作的规则。
2. 策略可以是确定性的，也可以是概率性的。
3. 价值函数  $V(s)$  衡量从状态  $s$  出发能获得的期望回报。
4. 状态价值高，说明从这个状态出发更有希望获得高累计奖励。
5. 最优策略是在长期意义下使期望累计奖励最大的策略。
6. 最优价值函数和最优策略相互关联。

易错点：

1. “策略就是状态到动作选择的规则。”正确。
2. “价值函数只看当前一步奖励，完全不看未来。”错误。
3. “最优策略追求最大化期望累计奖励。”正确。

## 8.7 MDP 与马尔可夫性

复习重点：

1. MDP 是 Markov Decision Process，马尔可夫决策过程。
2. MDP 是强化学习常用数学模型。
3. 马尔可夫性指未来只依赖当前状态和动作，而不依赖完整历史。
4. 如果当前状态已经包含足够信息，就不需要再记住所有过去信息。
5. MDP 常包含状态、动作、转移概率、奖励、折扣因子等元素。

必背句：

马尔可夫性：给定当前状态和动作后，未来状态的概率分布与更早历史无关。

易错点：

1. “MDP 是马尔可夫决策过程。”正确。
2. “马尔可夫性要求未来依赖全部历史。”错误。
3. “当前状态如果足够完整，就可以用于预测下一状态分布。”正确。

## 8.8 Bellman 方程的直观含义

复习重点：

1. Bellman 方程刻画当前价值和未来价值的递推关系。
2. 当前状态的价值等于当前奖励加上折扣后的下一状态价值的期望。
3. Bellman 方程是许多强化学习算法的基础。
4. 不要求推导复杂公式，只要理解“当前价值 = 当前收益 + 未来价值”。

直观形式：

当前价值 = 当前奖励 +  $\gamma$  × 未来价值

不考虑动作时，向量形式：

$$V = R + \gamma P V$$

易错点：

1. “Bellman 方程把当前价值和未来价值联系起来。”正确。
2. “Bellman 方程完全不考虑未来。”错误。
3. “Bellman 方程是强化学习中的重要基础。”正确。

## 8.9 DP、MC 与深度强化学习

复习重点：

1. 动态规划 DP 可以利用 Bellman 方程迭代求价值。
2. DP 的优点是可以较精确计算价值。
3. DP 的缺点是通常需要知道环境转移概率 P。
4. 蒙特卡洛 MC 通过多次实验采样取平均。
5. MC 的优点是简单直接，不一定需要明确知道转移概率。
6. MC 的缺点是随机性大，方差可能大。
7. 深度强化学习把深度神经网络引入强化学习。
8. 深度神经网络可以近似价值函数或策略。
9. DQN 用神经网络替代表格来拟合 Q 值。
10. 深度强化学习能处理高维状态，如图像输入。
11. 深度强化学习也带来训练不稳定、数据需求大、算力需求高等问题。

对照表：

| 方法  | 核心思想           | 优点       | 缺点           |
|-----|----------------|----------|--------------|
| DP  | 用 Bellman 方程迭代 | 可较精确计算   | 需要环境模型/转移概率  |
| MC  | 多次采样取平均        | 简单直接     | 随机性大，方差可能大   |
| DRL | 用深度网络近似价值或策略   | 处理高维复杂状态 | 训练难、数据和算力需求大 |

易错点：

1. “DP 通常需要知道转移概率。”正确。
2. “MC 通过采样估计价值。”正确。
3. “DQN 用神经网络近似 Q 值。”正确。
4. “深度强化学习完全没有训练困难。”错误。

## 8.10 强化学习应用

复习重点：

1. 强化学习可用于游戏 AI。
2. 强化学习可用于机器人控制。
3. 强化学习可用于自动驾驶、交通灯调度、推荐系统、数据中心节能等。
4. 大模型后训练中也可使用强化学习。
5. 在 ChatGPT 一类系统中，可以用人类偏好构造奖励信号。
6. 在推理模型中，强化学习可用于增强推理能力。

易错点：

1. “强化学习只能玩游戏。”错误。
2. “强化学习可以用于机器人控制。”正确。
3. “大模型后训练中可能使用强化学习。”正确。

## 8.11 思考题

1. 判断：强化学习中，智能体通过与环境交互，并根据奖励反馈学习。  
答案：正确。
2. 填空：强化学习的目标通常是最大化期望 \_\_\_\_\_ 奖励。  
答案：累计。
3. 计算：未来三个奖励为 1, 0, 2,  $\gamma = 0.5$ , 折扣回报为 \_\_\_\_\_。  
答案： $1 + 0.5 \times 0 + 0.5^2 \times 2 = 1.5$ 。
4. 选择：尝试一个不确定但可能更好的动作，属于：A. 探索 B. 利用 C. 监督学习 D. 归一化。  
答案：A。
5. 判断：马尔可夫性指给定当前状态和动作后，未来状态分布不再依赖更早历史。  
答案：正确。

## 9 最后一遍串讲清单

### 9.1 残差网络必会

1. 深层普通网络可能出现退化问题。
2. 退化问题指网络加深后训练误差或测试误差反而变差。
3. 深层网络还可能遇到梯度消失和梯度爆炸。
4. 残差连接的基本形式是  $F(x) + x$ 。
5. 残差连接帮助信息和梯度在深层网络中传播。
6. ResNet 主要用于缓解深层网络训练困难和退化问题。

### 9.2 CNN 必会

1. 图像具有局部关联性和一定不变性。
2. 关联性会随距离衰减，课件中有指数衰减和幂级数衰减两类模型。
3. 幂级数衰减可以理解为由许多指数衰减成分的叠加，长程关联更不容易完全消失。
4. CNN 与 MLP 的核心区别是局部连接和参数共享。
5. 卷积核在图像上滑动，用局部窗口提取边缘、方向、纹理等局部特征。
6. 卷积核参数通常随机初始化，然后训练得到。
7. RGB 输入有 3 个通道；输出通道数由卷积核个数决定。
8. 会根据输入大小、卷积核大小、padding、stride 计算输出空间尺寸；输出通道数等于卷积核个数。
9. 会根据卷积核大小、输入通道数、卷积核个数计算卷积层参数量。
10. 池化可以降低空间尺寸，逐渐处理更长范围的关联，通常不改变通道数。
11. CNN 可由卷积、激活、池化、展平、全连接、输出层等模块按任务需要组合。

### 9.3 RNN/LSTM 必会

1. NLP 是处理自然语言的任务。
2. 语言有顺序、上下文、长程依赖。
3. Tokenization 把文本切分成 token。
4. One-hot 稀疏且不表达语义相似。
5. Embedding 可训练，能学习词向量。
6. Next Token Prediction 是预测下一个 token。
7. RNN 能处理序列，因为有历史状态。
8. BPTT 训练 RNN，长序列可能梯度消失/爆炸。
9. LSTM 通过记忆状态和门控机制缓解长程依赖问题。

### 9.4 Transformer 必会

1. Transformer 核心是注意力机制。
2. Embedding 把 token 变向量。
3. 位置编码提供顺序信息。
4. Q/K/V 分别是 Query/Key/Value。

5. Q 和 K 算相关性，V 被加权求和。
6. Softmax 把注意力分数变成权重。
7. Causal mask 防止 Decoder 看未来。
8. LayerNorm 逐 token 归一化。
9. FNN 逐 token 作用。
10. 输出层映射到词表大小。

## 9.5 训练现象必会

1. 频率原则：先低频后高频。
2. 低频是轮廓、平滑、整体趋势。
3. 高频是细节、振荡、快速变化，也可能包含噪声。
4. 早停可以减少拟合高频噪声。
5. 初始化影响训练。
6. 小初始化可能导致参数凝聚。
7. 参数凝聚使多个神经元趋向相似偏好。
8. 平坦解通常更鲁棒、泛化更好。

## 9.6 大模型必会

1. 大模型又叫基础模型。
2. 大模型通常参数量大、数据规模大、适应任务多。
3. Encoder-only 常用于理解任务。
4. Decoder-only 常用于生成任务。
5. 当前主流生成式大语言模型多是 Decoder-only。
6. 预训练学习基础能力。
7. 微调用特定任务数据继续训练。
8. 提示词工程改变输入，不一定改参数。
9. 知识蒸馏把大模型知识迁移到小模型。
10. 上下文学习不更新权重。
11. 思维链生成中间推理步骤。
12. 大模型可能产生幻觉和偏见。

## 9.7 强化学习必会

1. 强化学习是决策型任务。
2. 基本循环：状态、动作、奖励、下一状态。
3. 目标是最大化期望累计奖励。
4. 状态是完整环境信息，观察是智能体看到的信息。
5. 动作空间可以离散或连续。
6. 折扣因子  $\gamma$  在 0 到 1 之间。
7.  $\gamma$  越大越重视长期奖励。
8. 情节型任务有终点，连续型任务没有固定终点。
9. 探索是尝试未知，利用是使用已有知识。
10. 策略是选择动作的规则。
11. 价值函数衡量状态的期望回报。
12. MDP 是马尔可夫决策过程。
13. 马尔可夫性：未来只依赖当前状态和动作。
14. Bellman 方程联系当前价值和未来价值。
15. DP 需要环境模型，MC 通过采样估计。
16. 深度强化学习用神经网络近似价值函数或策略。

## 10 不建议作为考试重点的内容

以下内容可以课堂提到，但不建议作为选择、判断、填空题重点。

1. 卷积平移等变性的严格证明。
2. Pooling 反向传播细节。
3. LSTM 门控公式完整推导。
4. 正余弦位置编码、RoPE、相对位置编码的公式推导。
5. 多头注意力复杂矩阵维度计算。
6. 多尺度神经网络、Fourier 特征、NeRF。
7. Hessian、稳定边缘的数学推导。
8. LoRA、MCP、Agent、RAG 的技术细节。
9. TD、Sarsa、Q-learning 的公式推导。
10. Kaggle 注册、实验提交、代码复现操作。

## 11 两节复习课建议讲法

### 11.1 第一节课：网络结构主线

建议顺序：

1. 5 分钟：说明考试范围和“不考阅读材料”。
2. 10 分钟：残差网络，重点讲退化问题、梯度传播困难和残差连接。
3. 30 分钟：CNN，按“图像数据特点 -> 为什么需要卷积 -> 卷积/padding/stride/池化计算 -> 经典结构”讲。
4. 20 分钟：RNN/LSTM，重点讲序列、BPTT、梯度问题、门控机制。
5. 25 分钟：Transformer，重点讲 embedding、位置编码、QKV、mask、输出层。

课堂建议：

1. CNN 部分一定要带学生完整算 2-3 个尺寸题。
2. Transformer 部分不要推矩阵公式，重点讲 Q/K/V 的角色。
3. RNN/LSTM 部分不要推门控公式，重点讲为什么需要记忆和门。

### 11.2 第二节课：训练现象、大模型、强化学习

建议顺序：

1. 20 分钟：频率原则、早停、初始化、凝聚、平坦解。
2. 20 分钟：大模型架构、预训练、微调、提示词、蒸馏、scaling law、上下文学习、思维链。
3. 35 分钟：强化学习基本概念、回报计算、探索利用、MDP、Bellman、DP/MC/DRL。
4. 15 分钟：串讲易错点。

课堂建议：

1. 训练现象部分用“先学轮廓再学细节”来讲频率原则。
2. 大模型部分重点做概念辨析，不要展开最新技术细节。
3. 强化学习部分要反复画“状态-动作-奖励-下一状态”的循环。
4. Bellman 方程只讲“当前价值 = 当前奖励 + 折扣未来价值”。